

StreamKnight: Stream Tool Injection and Monitoring Technique

Mayank Singh Jadon

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
mayank.ms.j@gmail.com

Krishna Nishad

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
krishnanishad52513@gmail.com

Akshay Tiwari

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
akshay2003tiwari@gmail.com

Amrita Bhatnagar

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
bhatnagaramrita@akgec.ac.in

Abstract—When AI models are required to use multiple MCP servers, token usage increases significantly, and incorrect tool arguments may occur during execution. To address this problem, we propose the Stream Tool Injection and Monitoring Technique, an architecture that constructs an explicit tool route and monitors every tool call during execution. The technique reduces token overhead by exposing only the required tool at each step and improves reliability by validating arguments before execution. Unlike existing guardrails that focus primarily on output moderation, our approach targets tool usage itself. We evaluate our system through a direct comparison with standard MCP execution and demonstrate substantial token reduction and improved execution reliability.

Keywords—Conversational AI, Guardrails, Model Context Protocol

I. INTRODUCTION

Conversational AI relies heavily on external tools to perform various tasks. In November 2024, Anthropic released the Model Context Protocol (MCP) as an open standard for how AI integrates with external tools and data sources.

Although this is a good move, it also comes with some problems. When we give an AI a task to use MCP servers, it works well, but it uses too much processing, which makes it expensive to run, especially if you have many MCP servers. Also, sometimes during work, AI uses arguments in the wrong way. For example, in an argument that requires gmail, it uses example@gmail.com, which is not correct.

Since it is a new technology, existing guardrails such as NeMo Guardrails focus more on security than verifying whether the tool-calling process is working correctly [1].

To solve this issue, we propose **Stream Tool Injection And Monitoring Technique**, a system developed specifically to address this problem.

II. LITERATURE REVIEW

A. Risks of LLM Usage

LLM usage comes with its own risks, and it's important for us to understand this and tackle them as soon as possible. These risks are mainly [2], [3]:

- **The Model:** Models sometimes hallucinate and give out harmful or sensitive output.
- **Training Data:** Training data can be biased or incomplete and lacking sufficient knowledge.

It is important to see how these risks affect the way AI uses tools so that we can minimize any potential harm.

1) *The Model:* Current Large Language Models have some problems that cannot be fixed easily. Researchers are currently working to solve these problems.

Bias and Fairness: One problem is the bias of AI. They sometimes favor one group over another. This bias occurs because these models are trained on internet data and, if those data contain bias, the models also become biased [4]. To check for bias, researchers use two approaches: intrinsic bias evaluation, which looks inside the model to understand bias, and extrinsic bias evaluation, which checks what the model actually says during task completion.

Privacy and Copyright Risks: Large Language Models sometimes remember exact data in which they were trained, and this can cause the leakage of personal information [5].

Explainability Problems: Today, models are extremely large, containing trillions of parameters. They are also called black box because we cannot see what is happening inside them. This makes it difficult to trust these models in law, healthcare, or any other sensitive sector [6].

Non-Reproducibility: Large Language Models do not always give the same answer for the same question. They use randomness, statistics, and probability, which decreases the chance of producing identical answers [5].

2) *Data-Centric and Training-Related Threats:* Apart from these problems, data-related issues also happen in Large Language Models.

Dataset Poisoning: This means that harmful data are put in the training dataset. In this case, the model can learn biased or harmful information and will reproduce it during chat. Some attackers create a "backdoor" using data poisoning to get sensitive data out after the Large Language Model training is completed [7].

Hard Labels and Hallucination: Another problem comes from hard labeling of data. In this, a model is told that there is only one correct word with 100% probability and others are wrong. Because of this, the model can give an incorrect result with full confidence, also known as hallucination [8].

B. Architectures of LLM Guardrails

To address these problems, researchers came up with the idea of **guardrails**. These systems monitor and correct problems before they cause any harm or sensitive outputs [2].

1) *Foundational Approaches: Alignment and Moderation:* The most basic safety mechanism is to either train Large Language Models to follow rules or create systems that enforce those rules.

Model-Embedded Alignment: In this, the model is taught to follow rules during training. There are two methods for this, Reinforcement Learning through Human Feedback (RLHF) [9] and Constitutional AI. In RLHF, people give feedback to the model based on its responses to make it follow rules. In Constitutional AI, the Large Language Model (LLM) follows a set of rules. These approaches work but are not eco-friendly, cost a lot of money, and sometimes AI can give harmful or sensitive data by using some jailbreak techniques.

External Content Moderation Systems: One way is to use an external tool for checking. Models are like a black box. We cannot see what is happening inside the model; it is difficult to see its internal working [10].

2) *Programmable and Layered Frameworks:* Since previous methods were not good enough, researchers came up with a better idea, to use programmable guardrails to solve the issue.

Programmable Guardrails (NeMo Guardrails): NeMo Guardrail is one of the guardrail frameworks; it is created by NVIDIA and uses a programmable approach [1]. Its main functionalities are:

- **Topical Rails:** This keeps AI focused on the main topic rather than drifting from it. This keeps the guardrail on the main point.
- **Execution Rails:** This ensures that AI responses are correct and AI is not hallucinating.

Layered Protection Model: This means, instead of using a single system, using multiple safety layers to tackle problems. This decreases the chance of failure [2], mainly consist of three layers:

- **GateKeeper Layer (Outside):** Blocks harmful or tricky user inputs that can cause jailbreak.
- **Knowledge Anchor Layer (Middle):** Connects AI to trustworthy knowledge sources, reducing the risk of failure.
- **Parametric Layer (Inside):** Makes adjustments to the model during training.

3) *Advanced and Specialized Paradigms:* As Large Language Models are now being accepted and widely used in many areas, especially in sensitive ones, new guardrail systems are being proposed to safeguard it from jailbreak **Trust-Based Guardrails**.

Trust-Based Guardrails: Instead of treating all users equally, these systems use a “Trust Score,” calculated by [11]:

- **User History:** Past activity of the user.
- **Verified Identity:** The user’s occupation or position.

By using this score, guardrails can decide whether to allow a specific output or restrict it.

C. Hallucination: Causes and Fixes

LLMs sometimes hallucinate, means that they can produce output that is wrong, but LLMs show full confidence that it is right. This can create serious problems, especially in areas where accuracy is important. Hallucination can occur even in Retrieval Augmented Generation (RAG) systems [12].

1) *Types of Hallucinations:* **Faithfulness Hallucination:** When the output does not match the retrieved source [8]. **Factuality Hallucination:** When the output is factually incorrect in the real world [13].

2) *Why Hallucinations Happen in RAG:* Retrieval problems can occur due to [12]:

- Bad data sources.
- Query issues.
- Retriever inefficiency.
- Poor strategy design.

3) *How to Fix Hallucinations:*

- **RAG:** Use external verified information sources [12].
- **Knowledge Distillation:** Train smaller models using large, accurate models as teachers [14].
- **Self-Correction:** Verify answers using methods like Chain-of-Verification.
- **Agentic Frameworks:** Use multiple agents to cross-check outputs [15].
- **FINCH-ZK:** Fix incorrect outputs using validated external data [16].

D. The Model Context Protocol (MCP): A New Way for AI to Work Safely

The Model Context Protocol (MCP) was created by Anthropic in November 2024. Provides a standardized way for AI systems to create and use external tools safely. This allows AI to interact with the real world, which can also introduce risks if not properly controlled [17].

1) *How MCP Works:* MCP gives AI a structured way to communicate with external environments.

Client-Server Setup: MCP works in a client-server architecture. The client is an application, and the server holds the tools and data that the AI can access. They communicate using JSON-RPC 2.0 messages, either locally (STDIO) or over the internet (StreamableHTTP or SSE).

What MCP Can Do:

- **Server Features:** It gives AI access to external resources, tools, and prompts.
- **Client Features:** Handle how data is taken, how users ask for information, and what information users are allowed, mainly safety restrictions to limit access.

2) *Why MCP is Useful:* Depending on where you are in the ecosystem, MCP can have a range of benefits.

- **Developers:** MCP reduces development time and complexity when building, or integrating with, an AI application or agent.
- **AI applications or agents:** MCP provides access to an ecosystem of data sources, tools, and apps that will enhance capabilities and improve the end-user experience.
- **End-users:** MCP results in more capable AI applications or agents which can access your data and take actions on your behalf when necessary.

3) *Security Problems and How to Stay Safe:* MCP also introduces new risks as it allows AI to interact with the physical world [18].

Possible Attacks:

- Token Theft
- Server Hacking
- Prompt Injection
- Supply Chain Problems

Safety Rules:

- User control over MCP flow
- Data privacy and encryption
- Safe tool design
- Validation of model outputs
- Logging and auditing for traceability

E. The Fragility of Safety: How Fine-Tuning Affects Guardrails

Language models are sensitive to losing their safety mechanisms if fine-tuned; this happens if we use similar data to train the model as it was trained on. To tackle that, we have to make sure that we are using different data to fine-tune AI than the training set.

1) *How Similar Data Breaks Safety:* Research shows that fine-tuning with data similar to previous training sets increases the risks of jailbreak. We have to make sure that we are using different data to fine-tune AI than the training set. This happens because the model gives more importance to new data than the previous data, which can be used to train AI [19].

2) *What Can We Do:*

- **Keep Safety Data Private:** Don't use public safety data that could be exploited.
- **Check Similarity Before Fine-Tuning:** Make sure that the datasets are sufficiently different from the original training data.

F. Summary

In this section, we studied the major risks related to Large Language Models, including model bias, privacy leakage, jailbreak, non-reproducibility of output, training dataset poisoning, and hallucination. We examined existing guardrail architectures such as model alignment, external AI moderation, programmable guardrails, layered protection architecture, and trust-based systems to control sensitive information leakage and unsafe behavior. This section also analyzed the reasons

for hallucination, mainly in Retrieval-Augmented Generation, and presented current methods to control it such as knowledge distillation, self-correction, and multi-agent validation. Finally, we learned about the Model Context Protocol (MCP) as a standard mechanism for safe tool usage and highlighted how fine-tuning on similar data can weaken safety systems, emphasizing the need for careful dataset selection to preserve safety.

III. PROBLEM STATEMENT

In Conversational AI, Model Context Protocol (MCP) enables models to use external tools and data sources. However, this comes with one problem. When we give AI a task to work on at most three MCP servers, it works great. But when the number goes up, AI starts to fail. We see two types of problems:

- 1) As MCP servers are Increased, token size increases that in the end increases the cost.
- 2) Sometimes AI uses tool Arguments in Wrong Way.

This research aims to address these problems by developing a guardrail system that can help AI complete **tool calling** and maintain logical execution until completion.

IV. METHODOLOGY

To solve this issue, we created three AI methods, it works like this: (1) defining a proper path before tool execution, (2) checking if the path is properly followed, and (3) checking tool calls during execution.

A. Defining a Proper Path

Our architecture has a dedicated AI for tool planning, *Tool Planner*, that operates before any tool call begins. When the user gives a query, the Tool Planner phase follows this structure as shown:

- 1) **Tool Discovery:** The Tool Planner is connected with every MCP server that is configured and extracts the complete tool inventory using the MCP `list_tools` protocol. For each tool, we extract:
 - Tool name (e.g., `gmail_send_message`)
 - Their description and functionality

These extracted data are given to the Tool Planner with the user query. Since the Tool Planner does not get any arguments/parameters, this saves tokens that can be wasted on reading arguments/parameters. Also, the Tool Planner does not store any conversation history; this means it always starts fresh. This also reduces the tokens that can be wasted on just reading the past conversation.

- 2) **Route Generation:** After getting the user query and information about all tools, it creates a structured path of tools that can be used by the main AI to complete the query. The format looks like this:

```
["github_search_repos",  
 "github_get_repo_details",  
 "linear_create_issue"]
```

- 3) **Token-Aware Execution:** We track exact token usage during planning via OpenAI API responses, measuring:

- Input tokens: Length of planning prompt + tool specifications
- Output tokens: Length of generated route
- Total planning overhead for performance analysis

This ensures that the main AI operates only on a predefined logical path. This method significantly reduces the number of tokens that can be wasted on getting arguments and on the main AI reading the tools for every tool call.

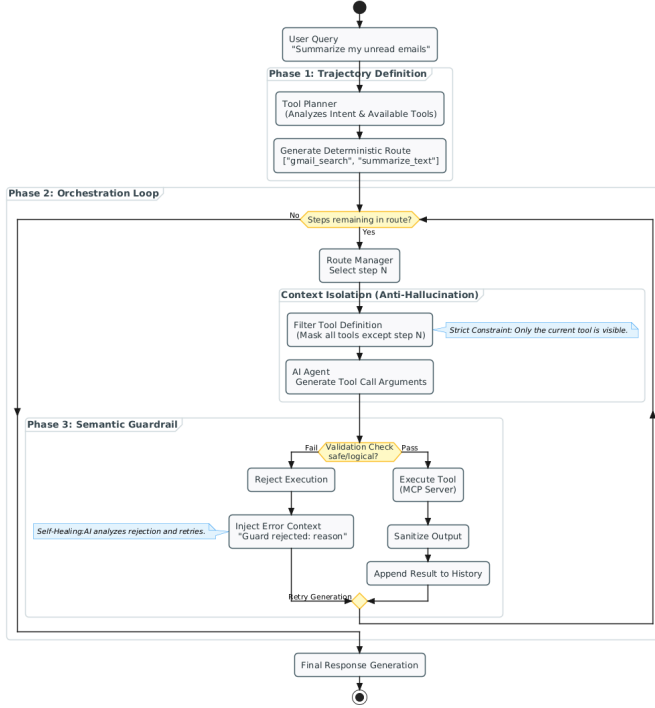


Fig. 1: Simplified workflow of the Stream Tool Injection And Monitoring Technique methodology.

B. Context Isolation and Path Enforcement

Once the path is decided by the Tool Planner, we implement strict context isolation to ensure that the main AI follows the planned route.

1) *Single-Tool Exposure Mechanism*: Our system employs a `RoutePointer` class that manages sequential tool access:

```
class RoutePointer:
    def __init__(self, route_list: List[str]):
        self.route = route_list or []
        self.step = 0 # Tracks current position in route

    def current(self) -> Optional[str]:
        """Returns the currently allowed tool name"""
        return self.route[self.step] if self.step < len(self.route) else None

    def advance(self):
        """Moves pointer to next tool in sequence"""
        self.step += 1
```

At each step of the query, the AI is only allowed to see the current tool required. The system filters out every tool from the inventory to expose only the tool currently allowed:

```
# Only expose the current tool from the planned route
allowed_tools = [t for t in all_tools if t["name"]
                 == pointer.current()]
response = openai_call(messages, allowed_tools) # AI
                 sees only one tool
```

2) *Token Efficiency Benefits*: Normally, if an AI has access to 50 tools and the user gives it a query like ‘read my emails and forward the summary to example.com’, the AI will first read the query, then extract all the tool details, their names, descriptions, and arguments of every tool, choose the correct tool to read the email, then for forwarding, it will again extract all 50 tools, read names, descriptions, and their arguments and choose the correct tool for writing the email and then send it. In this iteration, the AI is calling every tool twice, wasting too many tokens. Also, this will go into history, so it will fill the context length of AI too fast. If we increase the number of tools, it will just become worse.

However, our approach provides significant token efficiency improvements:

- **Reduced Context Size**: Since we are using a Tool Planner, it scans the full tool catalog once, meaning no wastage on seeing tools multiple times. This also saves the context window of the main AI as it sees only one tool at a time, significantly reducing the risk of filling the context window.
- **Focused Processing**: Since power is not wasted in gathering tool information, the AI focuses more on the main task.

C. Semantic Validation and Self-Correction

Our third layer of protection addresses the most common source of AI errors: incorrect tool usage. Even when following the planned path, AIs often provide wrong arguments, use invalid formats, or insert placeholder values (like example@gmail.com).

1) *Guard AI Validation*: We implement a dedicated *Guard AI* that validates every tool call before execution:

- **Schema Compliance**: Checks if arguments match the tool’s expected format and data types
- **Semantic Checking**: Validates that values make sense in context (proper emails, reasonable dates, appropriate content)
- **Structured Decisions**: Returns clear PASS/FAIL verdicts with specific reasons

2) *Self-Correction Through Feedback*: When the Guard rejects a tool call, it provides explicit feedback that enables the main AI to self-correct:

```
"Error: Guard rejected tool call:
Invalid email format. Please analyze and
retry with corrected arguments."
```

This creates a learning loop where the AI understands specific validation failures and improves its subsequent attempts, significantly reducing execution errors and improving output quality.

V. EXPERIMENTAL SETUP

A. Implementation Framework

We used **Python 3.11** to develop the framework. The system architecture relies on the following core components:

- **Model Engine:** All components (Tool Planner, Guard and Main AI) utilizes **GPT-4.1-mini**
- **Tool Integration:** We used **Klavis AI** MCP servers. These servers expose standardized interfaces for real-world applications. We used a total of 11 MCP servers, which include servers for GitHub, Attio, Slack, YouTube, HackerNews, Gmail, Linear, Google Calendar, Jira, Notion, and Google Drive, summing to a total of 88 tools.
- **Token Telemetry:** Token Usage was directly extracted from OpenAI API response header.

B. Evaluation Methodology

We used a comparative approach to test our system

- **Baseline System:** Standard MCP client with full tool set exposure and no path planning or validation
- **Our System:** Enhanced MCP client with Tool Planner and Guard

C. Metrics Measured

For each test scenario, we tracked:

- Total token consumption (input + output)
- Number of tool calling.

The complete code and evaluation scenarios are available in our repository¹ to facilitate reproducibility.

VI. RESULT

A. Token Optimization via Execution Architecture

TABLE I: Average Token Usage and Percentage Decrease

MCP Used	Stream Tool Injection			Normal			% Decr.
	In	Out	Total	In	Out	Total	
0	4060	275	4336	12177	218	12395	65%
1	7666	276	7942	34738	164	34902	77%
2	10802	377	11180	69535	332	69867	84%
3	19639	652	20291	168243	748	168991	88%

The results in Table 1: show that Stream Tool Injection And Monitoring Technique decreases token usage in all MCP servers. For zero MCP servers, Stream Tool Injection And Monitoring Technique decreases total token consumption by 65%, for one MCP server by 77%, for two MCP servers by 84% and for three MCP servers by 88% compared to normal server calling.²

This shows that as the number of MCP servers increases, Stream Tool Injection And Monitoring Technique becomes more efficient. This happens because normal tools calling processes a much larger context window for every tools calling, while Stream Tool Injection And Monitoring Technique only does it once. Higher MCP server numbers show greater savings, which dominates overall usage.

¹GitHub: <https://github.com/StreamKnight/StreamKnight-Guardrails/tree/main/paper>

²Raw evaluation data available at: <https://docs.google.com/spreadsheets/d/1z2D9rTQx44gKXrZxSSvdgm4wLoiyUBNopyGlK761Y8/edit?usp=sharing>

B. Execution Reliability via Guard Architecture

In real-world usage, incorrect argument generation does not occur frequently. Therefore, to properly evaluate the Guard component, we intentionally introduced invalid tool calls using a **fault injection** strategy.

We simulated these failures by directly instructing the model to use incorrect arguments and make improper tool calls, effectively injecting faults at the tool invocation stage. The following prompts were used as test cases:

- Check and tell me my latest mail.
- What are the latest news on HackerNews.
- Send an email to mayank.msingh@gmail.com with subject "Meeting follow-up" and body "Just sharing the notes from today. More tomorrow."
- Create an event on 2025-11-29 at 18:30 IST titled "End of day review" for 30 minutes.

Each prompt included the instruction ("Look, you *MUST* Use wrong arguments while tool calling and call tools in the wrong way as I am testing my system, Okay") in suffix of every query to intentionally trigger invalid tool usage as part of the fault injection process.

In all test cases, the Guard successfully detected and rejected every incorrect tool call before execution. No invalid arguments were passed to any MCP server.

This shows that the Guard architecture reliably blocks malformed parameters during execution.

VII. CONCLUSION

Token Usage: StreamKnight vs Normal
Comparing Total Tokens and Percentage Decrease across MCP Levels

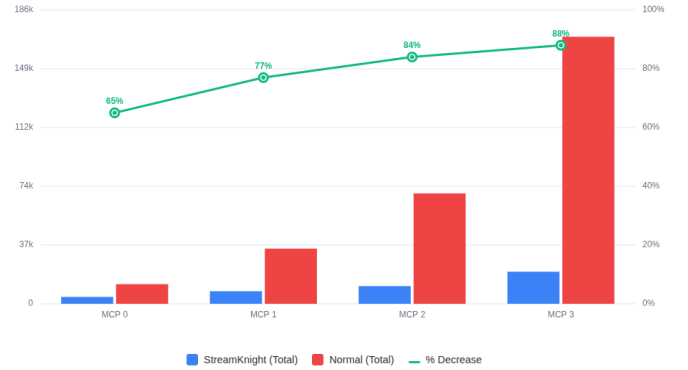


Fig. 2: Percentage decrease in token usage across MCP server categories.

This approach shows us decrease in operational cost, fast inference, and improved scalability for applications involving multiple MCP servers, and saves from accidental wrong tool calling. Fig. 2 shows that the advantage becomes more effective as complexity increases.

In general, the findings confirm that the stream tool injection and monitoring technique provides an optimized, resource-efficient, and scalable approach to executing MCP-based workflows.

REFERENCES

- [1] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2023, pp. 431–445.
- [2] X. Huang *et al.*, “A survey of safety and trustworthiness of large language models through the lens of verification and validation,” *Artificial Intelligence Review*, vol. 57, 2024.
- [3] S. Ghosh, P. Varshney, A. Padmakumar, J. R. Varghese, M. N. Sreedhar, T. Rebedea, and C. Parisien, “Aegis2.0: A diverse ai safety dataset and risks taxonomy for alignment of llm guardrails,” in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- [4] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021.
- [5] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] L. Weidinger *et al.*, “Taxonomy of risks posed by language models,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2022.
- [7] H. Liu, R. Kang, and X. Shen, “Exploring jailbreak attacks on llms through intent concealment and iterative prompting,” in *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- [8] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Findings of the Association for Computational Linguistics: ACL 2020*, 2020.
- [9] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano, “Learning to summarize with human feedback,” in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] D. Ganguli *et al.*, “Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned,” *Transactions on Machine Learning Research (TMLR)*, 2022.
- [11] Y. Bai *et al.*, “Constitutional ai: Harmlessness from ai feedback,” *Transactions on Machine Learning Research (TMLR)*, 2022.
- [12] W. Zhang and J. Zhang, “Hallucination mitigation for retrieval-augmented large language models: A review,” *Mathematics*, vol. 13, no. 5, p. 856, 2025.
- [13] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [14] K. Tian, E. Mitchell, H. Yao, C. D. Manning, and C. Finn, “Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] P. Manakul, A. Liusie, and M. J. F. Gales, “Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [17] Y. Qin *et al.*, “Toollm: Facilitating large language models to master 16000+ real-world apis,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [19] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, and P. Henderson, “Fine-tuning aligned language models compromises safety, even when users do not intend to!” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.